FACISCAN

(FACE RECOGNITION ATTENDANCE SYSTEM)

B. Tech Computer Science & Engineering, 8th semester

Synergy Institute of Engineering & Technology, Dhenkanal

Guide
Prof. Smruti Ranjan Dash

Submitted by:

Chandra Sekhar Thatoi
2101230077

Saranya Sahu
2101230054

Subhalaxmi Priyadarshini
2101230068

Himanshu Kumar Sahoo
2101230037

DECLARATION

We hereby certify that the project entitled **“FACISCAN” by Chandra Sekhar Thatoi - 2101230077, Saranya Sahu - 2101230054, Subhalaxmi Priyadarshini - 2101230068, and Himanshu Kumar Sahoo - 2101230037** in partial fulfillment of requirements for the award of degree of B.Tech. (Computer Science and Engineering) submitted in the Department of Computer Science & Engineering at SYNERGY INSTITUTE OF ENGINEERING & TECHNOLOGY, DHENKANAL under BIJU PATNAIK UNIVERSITY OF TECHNOLOGY, ROURKELA is an authentic record of their work carried out under the supervision of **Prof. Smruti Ranjan Dash**. The matter presented has not been submitted by them in any other University / Institute for the award of B.Tech. Degree.

Guide
(Computer Science & Engineering Department)

HOD
(Computer Science & Engineering Department)

External
Examiner

Acknowledgement

We would like to express our heartfelt gratitude to all those who have helped us in the completion of our project titled **FACISCAN *(Face Recognition Attendance System)*.**

First and foremost, we sincerely thank our project guide, **Prof. Smruti Ranjan Dash**, for their invaluable guidance, constant support, and encouragement throughout the course of this project. Their expertise and insightful suggestions were crucial in shaping the project and overcoming the various challenges we encountered.

We are also deeply grateful to the faculty members of the Department of Computer Science and Engineering at Synergy Institute of Engineering & Technology for their continuous support and motivation. Their vast knowledge and constructive feedback enriched our understanding of the subject matter and enhanced the quality of our work.

Our sincere thanks go to the libraries, research foundations, and online resources that provided access to research papers, books, and tools, particularly **OpenCV**, **Trinket,** and **Python libraries**, which were essential to the technical development of the system.

We would also like to acknowledge the valuable contributions from our peers and colleagues, who assisted with troubleshooting and provided feedback on various aspects of the project. Their inputs were instrumental in fine-tuning our system.

Saranya Sahu
2101230054

Chandra Sekhar Thatoi
2101230077

Subhalaxmi Priyadarshini
2101230068

Himanshu Kumar Sahoo
2101230037

CONTENTS

ABSTRACT 1

INTRODUCTION 2

PROBLEM STATEMENT 3

FEASIBILITY STUDY 4

OBJECTIVE 5

REQUIREMENT ANALYSIS AND SPECIFICATION 6

DESIGN 8

METHODOLOGY 14

IMPLEMENTATION & CODING 21

EXPERIMENT REQUIREMENTS 36

RESULT AND ANALYSIS 39

ADVANTAGES 41

LIMITATION 42

FUTURE PROSPECTS 43

CONCLUSION 44

REFERENCE 45

ABSTRACT

In today's fast-paced world, automation and efficiency are key factors in the effective functioning of any organization. Traditional methods of recording attendance, such as manual sign-ins or RFID card swiping, are often prone to manipulation, time-consuming, and susceptible to human error. To address these limitations, this project proposes and implements a Face Recognition Attendance System, an intelligent and contactless solution that automates attendance by scanning and identifying faces.

The system leverages computer vision and machine learning technologies to recognize and verify individuals based on their facial features. It primarily uses OpenCV, Haar Cascade Classifier for face detection, and the Local Binary Pattern Histogram (LBPH) algorithm for face recognition. The database stores facial images of enrolled individuals, and during operation, the system captures a live image, compares it against the stored dataset, and marks the attendance of the recognized individual. The entire process is seamless, quick, and ensures that only authorized individuals are recorded.

This project also ensures a user-friendly interface and real-time logging of attendance, which can be exported for reporting and analysis. The system is highly secure as it relies on biometric data, which is unique to each person, thus reducing the chances of impersonation or proxy attendance. Additionally, it is cost-effective and scalable for deployment in schools, colleges, offices, and other institutions.

Despite its many advantages, the system does face challenges such as variations in lighting, facial expressions, occlusion due to masks or accessories, and aging. These factors can affect recognition accuracy. However, with advancements in artificial intelligence and the integration of deep learning models like CNNs in future versions, these challenges can be effectively mitigated.

FACISCAN
(FACE RECOGNITION ATTENDANCE SYSTEM)

INTRODUCTION

The person in photos and in real time videos can be identified using facial recognition systems. It is a tier of biometric security. Other types of biometric security are voice, fingerprint and eye recognition. In real time we can use it to unlock phones, find missing persons, aid forensic investigations, help the blind, etc. The main aim of this project is to build an attendance system based on face recognition. Different systems can be used for face recognition like attendance systems with fingerprint scanners, RFID tags and readers, facial recognition and location-based attendance systems. The attendance system with a fingerprint scanner minimizes the issue of proxy attendance. The system by using an RFID reader is much faster but high chances to get proxy attendance. Attendance systems using face recognition provide authenticated data also proxy data entering chances are much less as compared to other systems. In the proposed system, at the time of enrolment, video is captured and images of students are stored through facial detection, recognition and recorded in a database. Face recognition is a computer vision technique that identifies or verifies a person from a digital image or video frame. It leverages machine learning and deep learning algorithms to analyze facial features such as the distance between the eyes, shape of the cheekbones, jawline structure, and contour of the lips and nose. Once the system is trained with facial data, it can compare real-time input with stored data to verify identity and mark attendance automatically. The proposed system involves a user-friendly interface integrated with a camera that captures the image of the person standing in front of it. The system processes the image using pre-trained models and compares it with the stored dataset. If a match is found, attendance is recorded in a database with the user's details and timestamp. This eliminates the need for manual supervision and ensures accuracy and transparency in attendance records. In real time, video of a student entering a classroom will be captured, face will be detected and matched with the dataset images, name & roll number of the present student will be displayed along with updating the attendance.

PROBLEM STATEMENT

To develop an automated attendance system using face recognition. Concept In a classroom with large number of students, it is a very tedious and time-consuming task to take the attendance manually. Therefore, we can implement an effective system which will mark the attendance of students automatically by recognizing their faces. The process of this face recognition system is divided into various steps, but the important steps are detection of face and recognition of face. Firstly, to mark the attendance of students, the image of students' faces will be required. This image can be snapped from the camera device, which will be placed in the classroom at a suitable location from where the whole classroom can be covered. This image will act as input to the system. For the effective face detection, the image needs to be enhanced by using some image processing techniques like grayscale conversion of image and histogram equalization. To identify the students sitting on the last rows neatly, the histogram equalization of image needs to be done. Manual attendance systems also require administrative oversight, which increases workload and reduces productivity. Even biometric systems like fingerprint scanners pose limitations in terms of hygiene, especially in the post-COVID era, where contactless systems are becoming a necessity. Moreover, these systems can experience performance degradation due to physical damage to biometric traits or system failures.

In educational settings, the problem is further amplified by large class sizes, frequent roll calls, and the need for real-time attendance tracking and reporting. Teachers or staff members are often required to spend several minutes per session taking attendance, reducing time that could be better spent on instruction or productivity. Additionally, retrieving and analyzing attendance data over time for reporting and decision-making remains a challenge with traditional systems.

Hence, there is a need to develop a real time operating student attendance system which means the identification process must be done within defined time constraints to prevent omission. The extracted features from facial images which represent the identity of the students have to be consistent towards a change in background, illumination, pose and expression. High accuracy and fast computation time will be the evaluation points of the performance.

FEASIBILITY STUDY

1. Technical Feasibility

The project is technically feasible as it relies on well-established technologies and tools. The system can be developed using open-source libraries like **OpenCV** and **face recognition** (built on dlib), which provide robust image processing and face detection capabilities. Python serves as the primary programming language due to its simplicity and extensive library support. The system requires a standard computer with a webcam, eliminating the need for specialized hardware.

2. Economic Feasibility

The system is economically viable as it minimizes hardware costs and uses free, open-source software tools. Institutions or organizations do not need to invest in expensive biometric scanners or RFID systems. The primary cost involves development time and potential database hosting if integrated with a cloud platform. In the long term, the project offers significant cost savings by reducing administrative work, manual errors, and fraudulent attendance practices.

3. Operational Feasibility

Operational feasibility assesses whether the system will function effectively within the intended environment. The proposed system is user-friendly and requires minimal training. Once implemented, users simply need to appear before a webcam, and the system automatically records their attendance. Administrative staff can easily monitor records, generate reports, and add or update user information through the interface. This reduces workload and enhances overall efficiency. Moreover, the touchless nature of the system makes it well-suited for environments concerned with hygiene and safety.

4. Legal and Ethical Feasibility

The system must comply with **data privacy regulations** such as GDPR (for global standards) or local data protection laws. Proper consent must be obtained from users before collecting facial data. The data should be securely stored and used only for the intended purpose. By implementing encryption, access controls, and transparent data policies, the system can be legally and ethically deployed.

OBJECTIVE

1. To identify the student faces accurately.
2. To mark the attendance automatically.
3. To reduce the time and the efforts required for manual attendance to provide a valuable attentive system for both teacher and students.

4. It provides flexibility and reduces the time loss. There will be no chance for a proxy. The objective of this project is to develop face recognition based automated student attendance system. Expected achievements in order to fulfill the objectives are:

• To detect the face segment from the video frame.
• To extract the useful features from the face detected.
• To classify the features in order to recognize the face detected.
• To record the attendance of the identified student.

REQUIREMENT ANALYSIS AND SPECIFICATION

Requirement analysis is a crucial phase in the software development lifecycle that identifies and documents what the system must do and how it should perform. For the **Face Recognition Attendance System (FRAS)**, the goal is to automate attendance using facial recognition, ensuring accuracy, speed, and ease of use. The system requirements are classified into two categories: **Functional Requirements** and **Non-Functional Requirements**.

1. Functional Requirements

These describe the specific functionalities the system must support:

- **User Registration**: The system must allow new users (students or employees) to register by capturing their facial image and storing it with associated information like name, ID, and department.
- **Face Detection and Recognition**: The system must capture a real-time image using a webcam and detect and recognize the user's face against the stored dataset.
- **Attendance Marking**: Once a face is recognized, the system should automatically log the attendance along with the current date and time.
- **Database Management**: A backend database must be maintained to store user details and attendance records securely.
- **Admin Panel**: Administrators should be able to add/remove users, view attendance logs, and generate reports.
- **Report Generation**: The system should generate daily, weekly, or monthly attendance reports for each user.
- **Error Handling**: If a face is not recognized or an error occurs, the system must prompt an appropriate message.

2. Non-Functional Requirements

These define the quality attributes of the system:

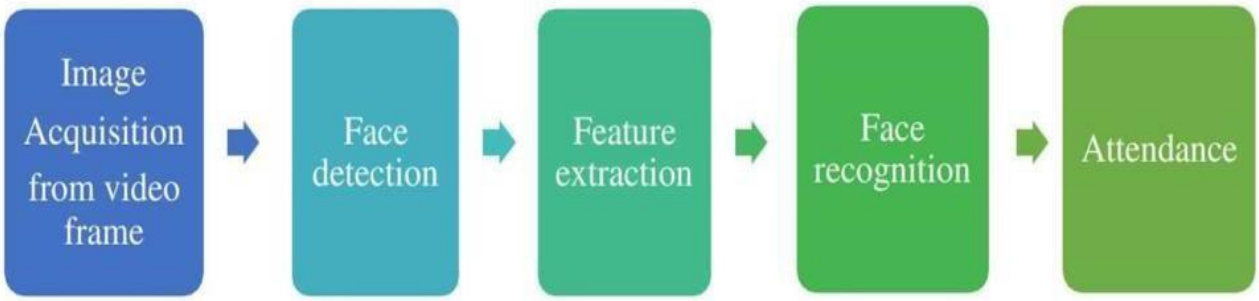
- **Performance**: The system must respond quickly (within 2–3 seconds) during face recognition and logging processes.
- **Scalability**: It should support a growing number of users without performance degradation.
- **Security**: All personal and facial data should be stored securely using encryption and access controls.
- **Usability**: The GUI should be intuitive and simple enough for non-technical users to operate.
- **Reliability**: The system should work consistently under normal usage and be able to handle common issues like minor lighting changes or partial face visibility.
- **Portability**: The software should be installable on standard desktop or laptop systems with basic camera setups.

3. System Requirements

- **Hardware**: Webcam, PC with minimum 8GB RAM and i5 processor or higher.
- **Software**: Python, OpenCV, face_recognition library, SQLite/MySQL, Tkinter (for GUI).

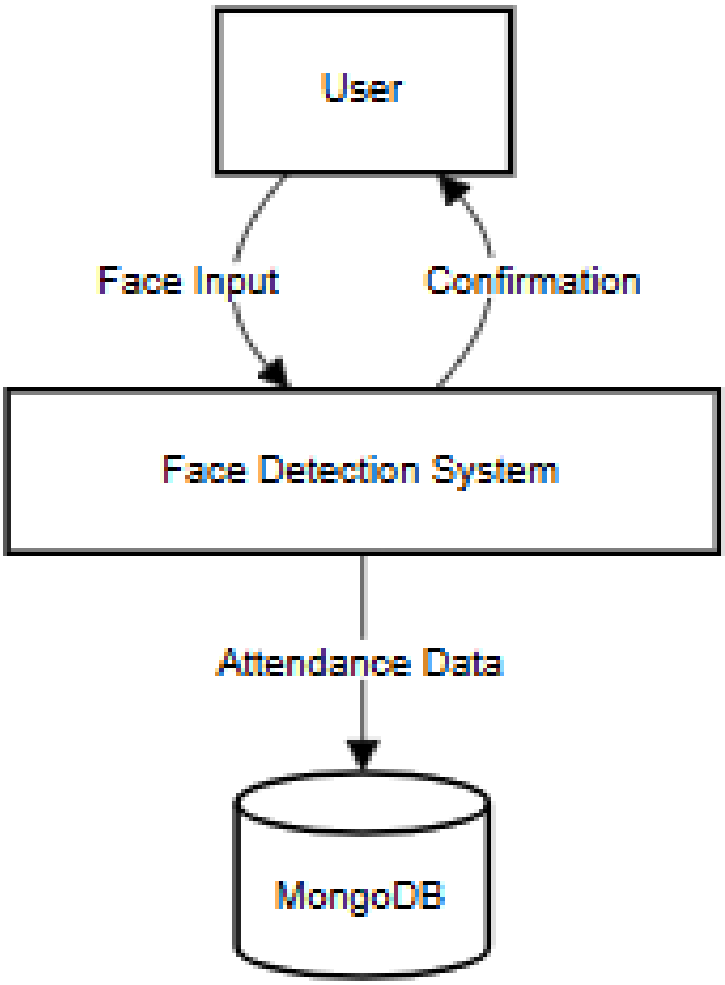
DESIGN

Architecture Model

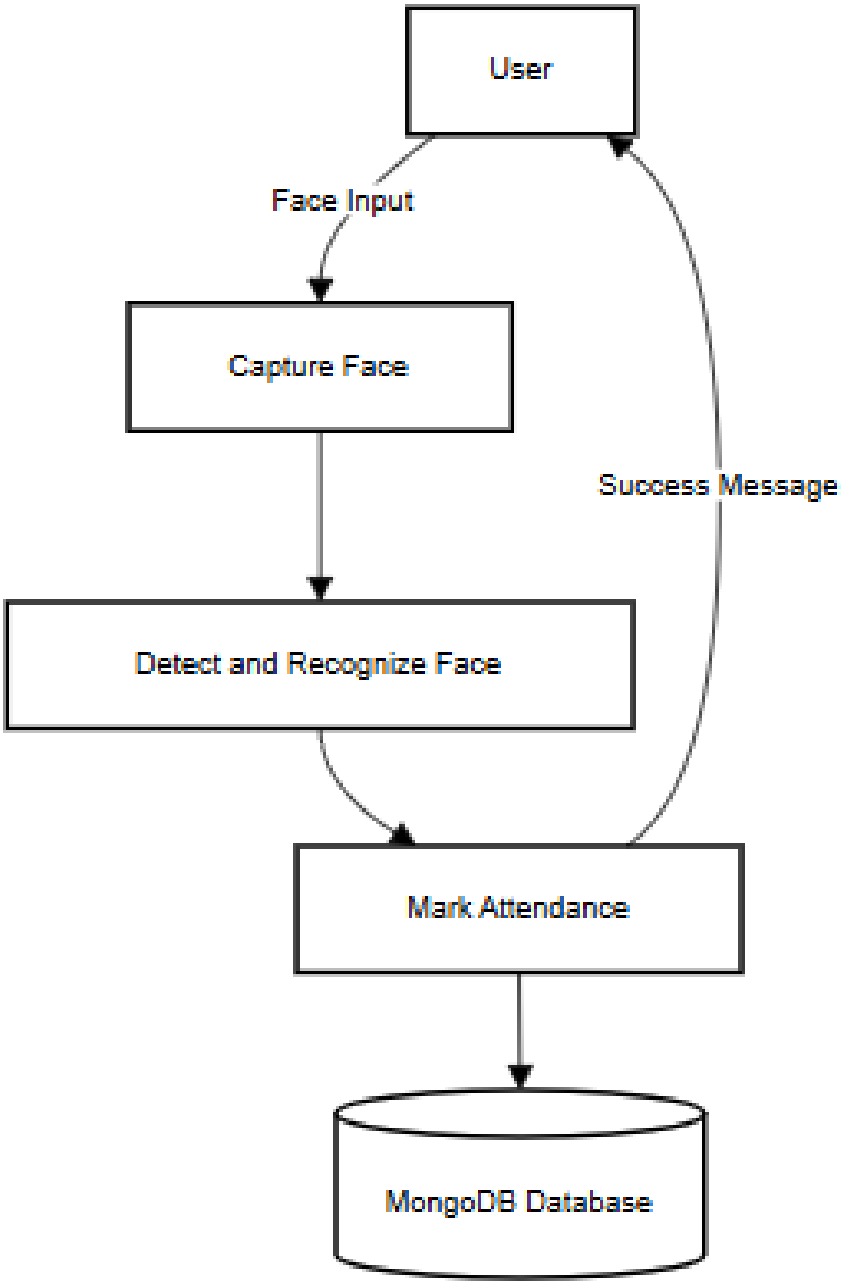


(Fig-1) Proposed system architecture

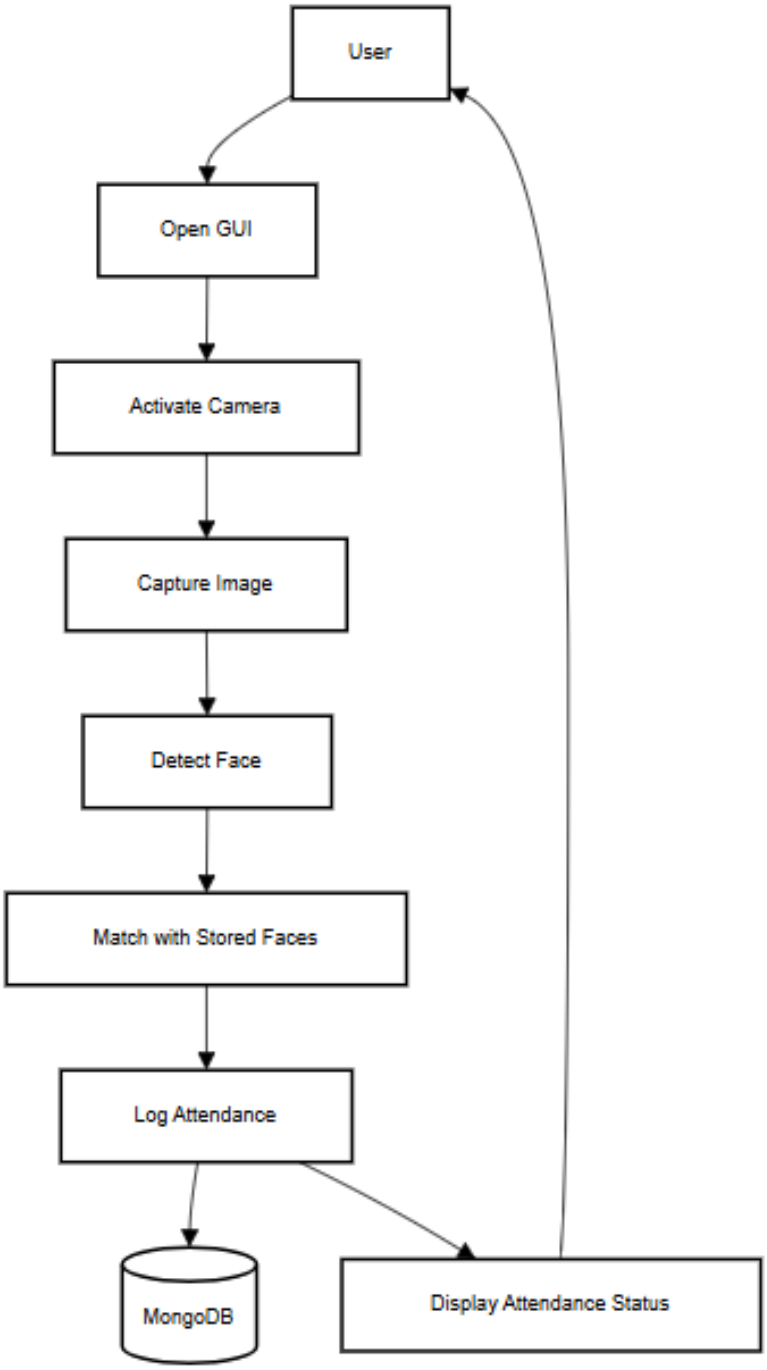
DFD Level-0


User
Face Input
Confirmation
Face Detection System
Attendance Data
MongoDB

DFD Level-1

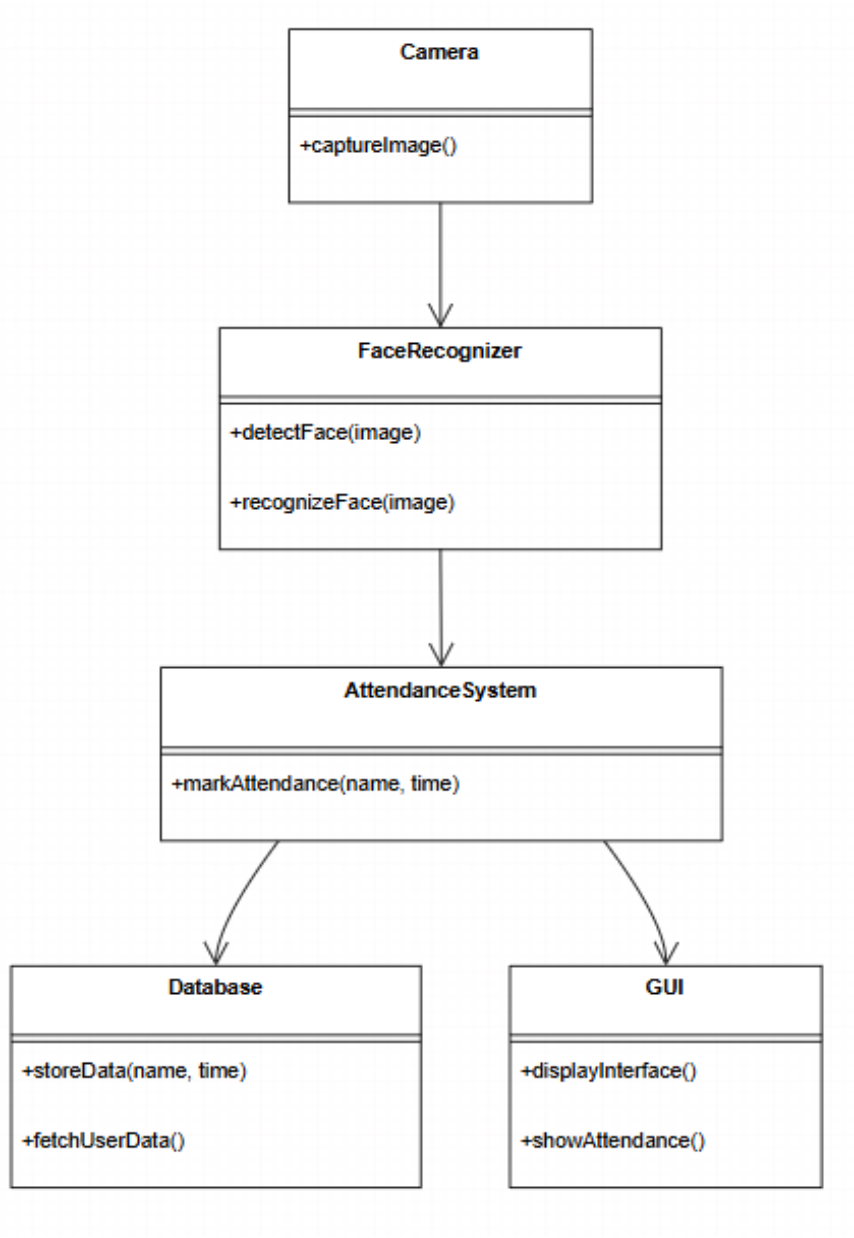

User
Face Input
Capture Face
Success Message
Detect and Recognize Face
Mark Attendance
MongoDB Database

DFD Level-2


User
Open GUI
Activate Camera
Capture Image
Detect Face
Match with Stored Faces
Log Attendance
MongoDB
Display Attendance Status

UML Diagrams

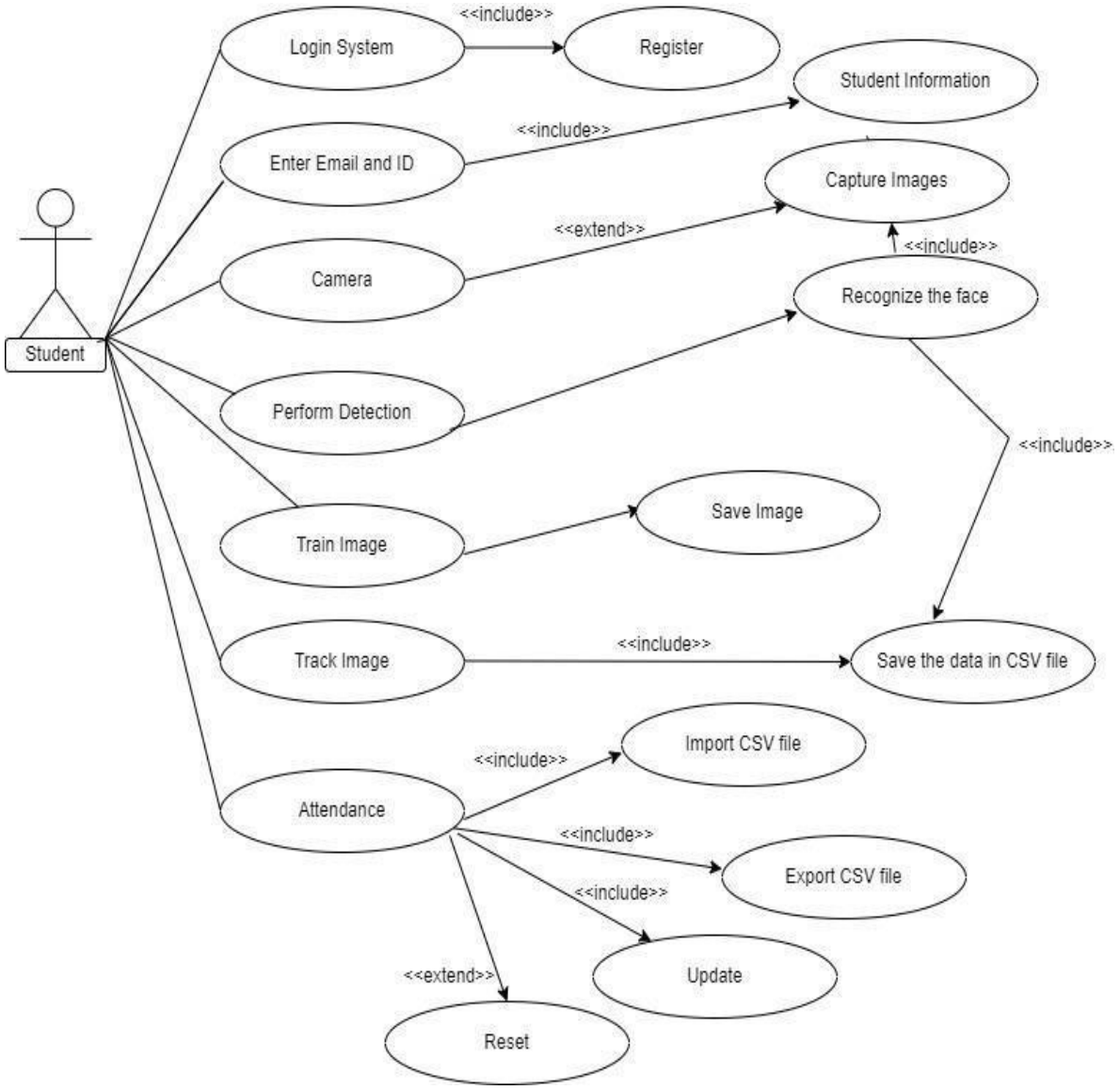
A UML diagram is a partial graphical representation (view) of a model of a system under design, implementation, or already in existence. UML diagram contains graphical elements (symbols) - UML nodes connected with edges (also known as paths or flows) - that represent elements in the UML model of the designed system.



(Fig-2) Uml diagram

Use Case Diagram

In Use-case diagrams model the behavior of a system and help to capture the requirements of the system. Use-case diagrams describe the high-level function and scope of a system. These diagrams also identify the interaction between the system and its actors, The use cases and actors in use-case diagrams describe what the system does and how the actor use it, but not how the system operates internally.



(Fig-3) Use case Diagram

METHODOLOGY

Proposed System

The proposed **Face Recognition Attendance System (FRAS)** is designed to automate the process of recording attendance using facial biometrics. The system uses computer vision and machine learning algorithms to detect and recognize faces, providing a contactless, efficient, and secure method of attendance tracking.

The development methodology follows a modular approach, divided into key functional stages:

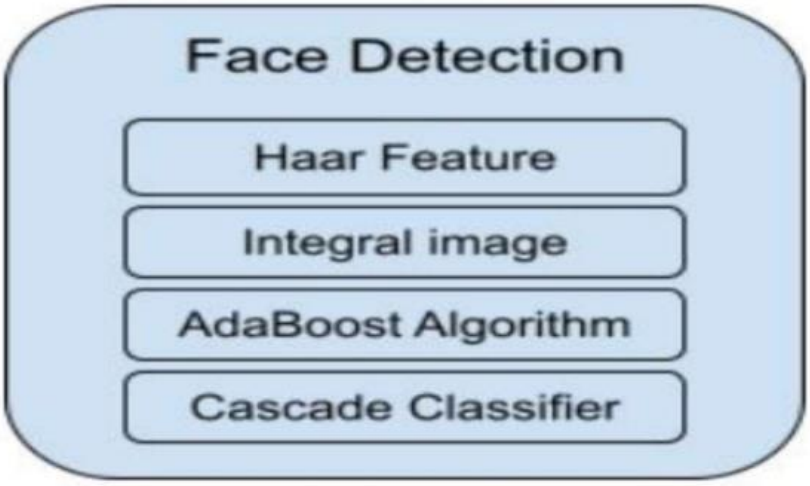
1. **Image Acquisition**: The system uses a webcam to capture real-time facial images of users. During the registration phase, multiple images of each user are taken from different angles and stored for training purposes.
2. **Preprocessing**: The captured images are converted into grayscale, resized, and filtered to enhance facial features and ensure consistency under varying lighting conditions. This step improves recognition accuracy.
3. **Face Detection and Encoding**: The system utilizes the face_recognition library to detect faces and extract unique facial features. These features are then encoded into numerical vectors, which are stored in a dataset for each registered user.
4. **Face Recognition and Matching**: During the attendance process, the webcam captures the user's face, encodes it, and compares it with the stored dataset using distance-based matching algorithms. If a match is found within a predefined threshold, the system records the user's attendance.
5. **Attendance Logging**: Upon successful recognition, the user's name, ID, date, and timestamp are recorded in a secure database (e.g., MySQL). The system avoids duplicate entries within a single session.
6. **User Interface and Admin Control**: A simple GUI is provided for users to register, and for administrators to manage users, view attendance records, and generate reports.

Haar Cascade Algorithm

It is a machine learning based approach where a cascade function is trained from a lot of positive and negative images (where positive images are those where the object to be detected is present, negative is those where it is not). It is then used to detect objects in other images.

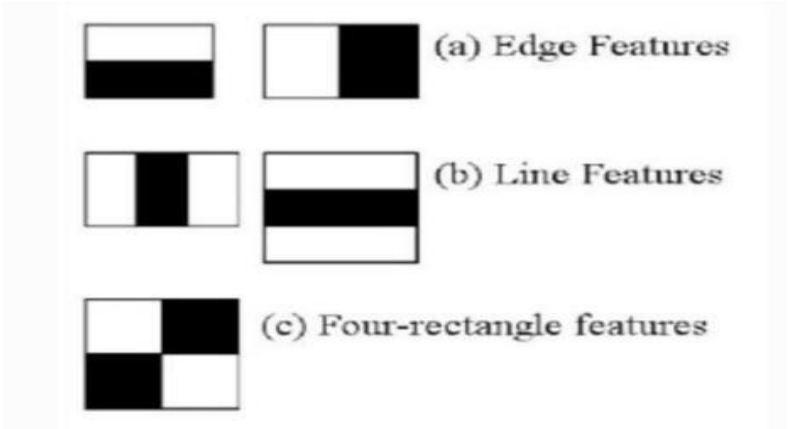
Luckily, OpenCV offers pre-trained Haar cascade algorithms, organized into categories (faces, eyes and so forth), depending on the images they have been trained on.

The algorithm can be explained in four stages:



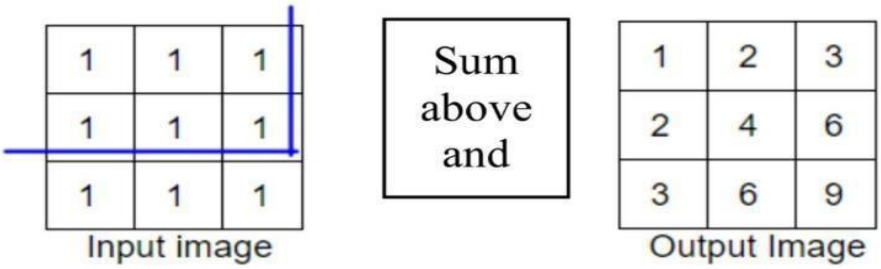
(Fig-4) Haar cascade Process

Haar features are the same as convolutional kernels and are used to detect the features in a given image. There are different kinds of Haar features such as line feature, edge feature, four - rectangle feature etc. The first step is to collect the Haar features. A Haar feature is essentially calculations that are performed on adjacent rectangular regions at a specific location in a detection window. The calculation involves summing the pixel intensities in each region and calculating the differences between the sums. Each feature is a single value obtained by subtracting sum of pixels under white rectangle from sum of pixels under black rectangle. Haar cascade algorithm makes use of 24*24 windows which ends up calculating 160000+ features in a window. To simplify the work of calculating the feature values, an Integral image algorithm is introduced. Here are some examples of Haar features below.

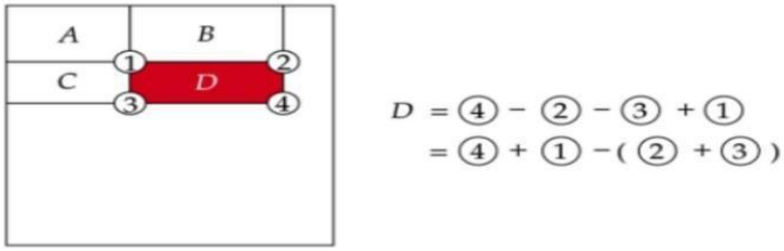


(Fig-5) Haar Feature

Creating Integral Images: Integral images essentially speed up the calculation of these Haar features. Instead of computing at every pixel, it instead creates sub-rectangles and creates array references for each of those sub-rectangles. These are then used to compute the Haar features. To reduce the computation of the pixels to find the feature values, Viola Jones introduced a technique called Integral Image. As shown in fig 4, the value of the pixel at (x, y) in an Integral image is calculated by adding the values of pixels above and to the left of (x, y) pixel.



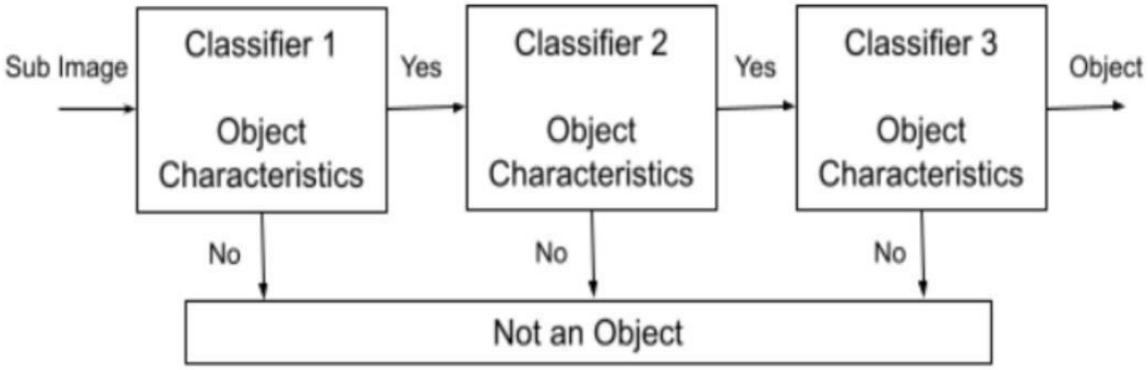
Calculation of the sum of all pixels inside any given rectangle can be done by using only four values at the corner of that rectangle with the help of Integral image.



Adaboost essentially chooses the best features and trains the classifiers to use them. It uses a combination of "weak classifiers" to create a "strong classifier" that the algorithm can use to detect objects. Weak learners are created by moving a window over the input image, and computing Haar features for each subsection of the image. This difference is compared to a learned threshold that separates non-objects from objects. Because these are "weak classifiers," a large number of Haar features is needed for accuracy to form a strong classifier. The last step combines these weak learners into a strong learner using cascading classifiers.

The cascade classifier is made up of a series of stages, where each stage is a collection of weak learners. Weak learners are trained using boosting, which allows for a highly accurate classifier from the mean prediction of all weak learners. Based on this prediction, the classifier either decides to indicate an object was found (positive) or move on to the next region (negative).

Stages are designed to reject negative samples as fast as possible, because a majority of the windows do not contain anything of interest. It's important to maximize a low false negative rate, because classifying an object as a non-object will severely impair your object detection algorithm.



(Fig-6) Cascade Classifier

Cascade classifiers comprise various stages. All the features are grouped into different stages and each stage contains a strong classifier and a feature. To design these stages, AdaBoost is used.

The window is discarded if it does not contain a face. While training a classifier the number of features, stages and thresholds are taken into account.

LBPH Algorithm for Face Recognition

The LBPH algorithm is based on the concept of **Local Binary Patterns (LBP)**, a texture descriptor that labels the pixels of an image by thresholding the neighborhood of each pixel and treating the

result as a binary number. LBPH builds upon this concept to create a robust feature vector that represents the local features of a face image.

1. Parameters

The LBPH uses 4 parameters:

i. Radius: the radius is used to build the circular local binary pattern and represents around the central pixel. It is usually set to 1.

ii. Neighbors: the number of sample points to build the circular local binary pattern. It is usually set to 8.

iii. Grid X: the number of cells in the horizontal direction. It is usually set to 8.

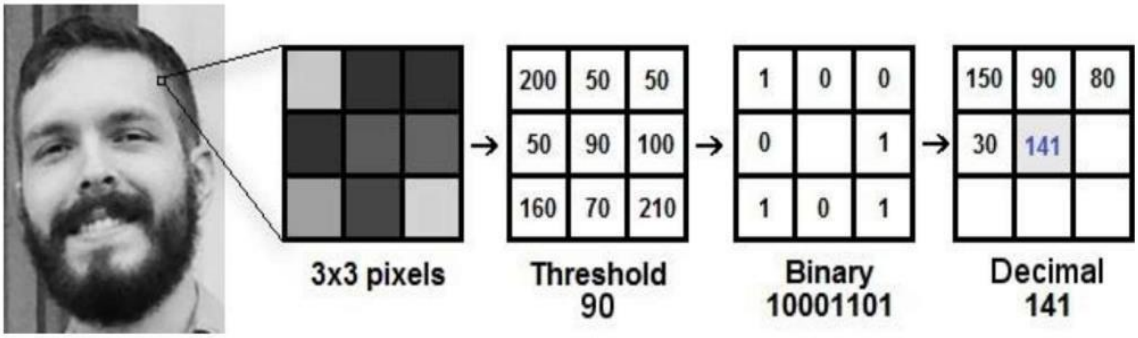
iv. Grid Y: the number of cells in the vertical direction. It is usually set to 8.

2. Training the algorithm:

To do this we need to use a dataset with the facial images of the people we want to recognize. We need to also set an ID (it may be a number or the name of the person) for each image, so the algorithm will use this information to recognize an input image and give you an output. Images of the same person must have the same ID.

3. Applying the LBP operation:

The first computational step of the LBPH is to create an intermediate image that describes the original image in a better way, by highlighting the facial characteristics. The algorithm uses a



concept of a sliding window based on the parameter's radius and neighbors.

(Fig-7) LBPH

At the end of this procedure, we have a new image which represents better the characteristics of the original image. Based on the image above, let's break it into several small steps so we can understand it easily:

i) Suppose we have a facial image in grayscale.

ii) We can get part of this image as a window of 3x3pixels.

iii) It can also be represented as a 3x3 matrix containing the intensity of each pixel (0~255).

iv) Then, we need to take the central value of the matrix to be used as the threshold.

v) This value will be used to define the new values from the 8 neighbours.

vi) For each neighbor of the central value (threshold), we set a new binary value. We set 1 for values equal or higher than the threshold and 0 for values lower than the threshold.

vii) Now, the matrix will contain only binary values (ignoring the central value). We need to concatenate each binary value from each position from the matrix line by line into a new binary value (e.g., 10001101). Note: some authors use other approaches to concatenate the binary values (e.g., clockwise direction), but the final result will be the same.

$$LBP(gp_x, gp_y) \sum_{p=0}^{P-1} S(gp - gc) \times 2^p$$

Uniform LBP formula

gc- the intensity value of the central pixel gp- the intensity of the neighbouring pixel with index p the function S can be expressed as:

$$s(x) = \begin{cases} 1 & if \quad x \geq 0 \\ 0 & if \quad x < 0. \end{cases}$$

threshold (step) function

P- number of sampling points on a circle of radius R (circular neighborhood). P- controls the quantization of the method.

R- determines the spatial resolution of the method or operator. Then, we convert this binary value to a decimal value and set it to the central value of the matrix, which is actually a pixel from the original image. At the end of this procedure (LBP procedure), we have a new image which represents better the characteristics of the original image.

4. Extracting the Histograms:

Divide face images into R (for example R = 8 x 8 = 64 Regions) local regions to extract LBP histograms. The LBP histograms extracted from each sub-region are used for calculation and combined into a single, histogram with spatial advanced features defined as:

$$H_{i,j} = \sum_{x,y} I\left\{f_l(x,y) = i\right\} I\left\{(x,y) \in R_j\right\}, i = 0, \ldots, n-1, j = 0, \ldots, m-1.$$

A histogram formula of the LBP

Where: n: is the number of different labels produced by the LBP operator. m: is the number of sub-regions. and I{…} is defined as follows:

$$I\{A\} = \begin{cases} 1, & \text{A is true} \\ 0, & \text{A is false.} \end{cases}$$

binary function

5. Performing the face recognition:

In this step, the algorithm is already trained. Each histogram created is used to represent each image from the training dataset. So, given an input image, we perform the steps again for this new image and creates a histogram which represents the image.

i) So, we find the image that matches the input image we just need to compare two histograms and return the image with closest histogram.

ii) We can use various approaches to compare the histograms. The search for the best match histogram can be also implemented by calculating the Euclidean distance between the new and available histograms:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \cdots + (p_i - q_i)^2 + \cdots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}.$$

$\boldsymbol{p}, \boldsymbol{q}$ = two points in Euclidean n-space

$\boldsymbol{q_i}, \boldsymbol{p_i}$ = Euclidean vectors, starting from the origin of the space (initial point)

$\boldsymbol{n}$ = n-space

iii) So, the algorithm output is the ID from the image with the closest histogram.

IMPLEMENTATION & CODING

Attendance.py

```
from flask import Flask, render_template, request, redirect, url_for, flash,
session,jsonify,send_from_directory

from models import db, User, Subject, Attendance

from werkzeug.security import generate_password_hash, check_password_hash

from datetime import datetime

import os

from face_recognition.add_face import add_face

from face_recognition.recognize_face import recognize_face


from flask import Flask

from models import db  # Import db from models


app = Flask(__name__)

from flask_login import LoginManager, UserMixin, login_user, logout_user, current_user,
login_required


# Add this after creating your app

login_manager = LoginManager()

login_manager.init_app(app)

login_manager.login_view = 'login'

from models import User, Subject, Attendance

app.config['SECRET_KEY'] = 'your-secret-key'

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///attendance.db'

app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

app.config['UPLOAD_FOLDER'] = 'data'
```

```
# Initialize database
db.init_app(app)

# Create tables
with app.app_context():
    db.create_all()
    # Add default admin if not exists
    if not User.query.filter_by(is_admin=True).first():
        admin = User(
            username='admin',
            name='Admin',
            registration_no='ADMIN001',
            is_admin=True
        )
        admin.set_password('admin123')
        db.session.add(admin)
        db.session.commit()
def get_id(self):
        return str(self.id)

# Add the user loader
@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))
@app.route('/')
def home():
```

```
    if 'user_id' in session:
        user = User.query.get(session['user_id'])
        if user.is_admin:
            return redirect(url_for('admin_dashboard'))
        else:
            return redirect(url_for('student_dashboard'))
    return redirect(url_for('login'))
import re
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        name = request.form['name']
        registration_no = request.form['registration_no']
        password = request.form['password']

        if User.query.filter_by(username=username).first():
            flash('Username already exists', 'danger')
            return redirect(url_for('register'))

        if not re.fullmatch(r'2101230\d{3}', registration_no):
            flash('Invalid registration number. It must be 10 digits starting with 2101230.', 'danger')
            return redirect(url_for('register'))

        user = User(username=username, name=name, registration_no=registration_no)
        user.set_password(password)
```

```
        db.session.add(user)
        db.session.commit()

        # After registration, redirect to face registration
        session['temp_user_id'] = user.id
        return redirect(url_for('register_face'))

    return render_template('auth/register.html')

@app.route('/register-face', methods=['GET', 'POST'])
def register_face():
    if 'temp_user_id' not in session:
        return redirect(url_for('register'))

    user = User.query.get(session['temp_user_id'])
    if not user:
        return redirect(url_for('register'))

    if request.method == 'POST':
        # Call face registration function
        face_data_path = os.path.join(app.config['UPLOAD_FOLDER'],
f"{user.registration_no}.pkl")
        success = add_face(user.registration_no, face_data_path)

        if success:
            user.face_data_path = face_data_path
            db.session.commit()
```

```
            session.pop('temp_user_id')
            flash('Face registration successful! Please login.', 'success')
            return redirect(url_for('login'))
        else:
            flash('Face registration failed. Please try again.', 'danger')

    return render_template('auth/register_face.html', user=user)

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        user = User.query.filter_by(username=username).first()

        if user and user.check_password(password):
            session['user_id'] = user.id
            if user.is_admin:
                return redirect(url_for('admin_dashboard'))
            else:
                return redirect(url_for('student_dashboard'))
        else:
            flash('Invalid username or password', 'danger')

    return render_template('auth/login.html')
```

```
@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))

# Admin Routes
@app.route('/admin/dashboard')
def admin_dashboard():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    user = User.query.get(session['user_id'])
    if not user or not user.is_admin:
        return redirect(url_for('login'))

    subjects = Subject.query.all()
    students = User.query.filter_by(is_admin=False).count()
    return render_template('admin/dashboard.html', user=user, subjects=subjects,
student_count=students)

@app.route('/admin/view-attendance')
def view_attendance():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    user = User.query.get(session['user_id'])
    if not user or not user.is_admin:
```

```
        return redirect(url_for('login'))

    subject_id = request.args.get('subject_id')
    date = request.args.get('date', datetime.now().strftime('%d-%m-%Y'))

    if subject_id:
        attendances = Attendance.query.filter_by(
            subject_id=subject_id,
            date=date
        ).all()
    else:
        attendances = []

    subjects = Subject.query.all()
    return render_template('admin/view_attendance.html',
                           user=user,
                           attendances=attendances,
                           subjects=subjects,
                           selected_subject=subject_id,
                           selected_date=date)

# Student Routes
@app.route('/student/dashboard')
def student_dashboard():
    if 'user_id' not in session:
        return redirect(url_for('login'))
```

```
    user = User.query.get(session['user_id'])
    if not user or user.is_admin:
        return redirect(url_for('login'))

    return render_template('student/dashboard.html', user=user)

@app.route('/student/submit-attendance', methods=['GET', 'POST'])
def submit_attendance():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    user = User.query.get(session['user_id'])
    if not user or user.is_admin:
        return redirect(url_for('login'))

    if request.method == 'POST':
        subject_id = request.form['subject_id']
        subject = Subject.query.get(subject_id)

        if not subject:
            flash('Invalid subject selected', 'danger')
            return redirect(url_for('submit_attendance'))

        # Verify face
        recognized_roll_no = recognize_face(user.face_data_path)
        print(user.face_data_path)
```

```
        if recognized_roll_no == user.registration_no:
            now = datetime.now()
            attendance = Attendance(
                user_id=user.id,
                subject_id=subject.id,
                date=now.strftime('%d-%m-%Y'),
                time=now.strftime('%H:%M:%S')
            )
            db.session.add(attendance)
            db.session.commit()
            flash('Attendance submitted successfully!', 'success')
        else:
            flash('Face verification failed. Please try again.', 'danger')

    subjects = Subject.query.all()
    return render_template('student/submit_attendance.html', user=user, subjects=subjects)
@app.route('/admin/add-subject', methods=['POST'])
def add_subject():
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': 'Unauthorized'}), 401

    user = User.query.get(session['user_id'])
    if not user or not user.is_admin:
        return jsonify({'success': False, 'message': 'Unauthorized'}), 401

    name = request.form.get('name')
    code = request.form.get('code')
```

```
    total_number = request.form.get('total_number')

    if not name or not code or not total_number:
        return jsonify({'success': False, 'message': 'All fields are required'}), 400

    if Subject.query.filter_by(code=code).first():
        return jsonify({'success': False, 'message': 'Subject code already exists'}), 400

    try:
        total_number = int(total_number)
    except ValueError:
        return jsonify({'success': False, 'message': 'Invalid number'}), 400

    subject = Subject(name=name, code=code, total_number=total_number)
    db.session.add(subject)
    db.session.commit()

    return jsonify({'success': True, 'message': 'Subject added successfully'})

# Additional Student Routes
@app.route('/student/attendance-history')
def attendance_history():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    user = User.query.get(session['user_id'])
    if not user or user.is_admin:
```

```
        return redirect(url_for('login'))

    page = request.args.get('page', 1, type=int)
    per_page = 10

    attendances = Attendance.query.filter_by(user_id=user.id)\
        .order_by(Attendance.date.desc(), Attendance.time.desc())\
        .paginate(page=page, per_page=per_page)

    return render_template('student/attendance_history.html',
                           user=user,
                           attendances=attendances)

# Error Handlers
@app.route('/admin/attendance-history')
def admin_attendance_history():

    page = request.args.get('page', 1, type=int)
    student_id = request.args.get('student_id')
    subject_id = request.args.get('subject_id')
    date = request.args.get('date')
    per_page = 10

    # Base query
    query = Attendance.query

    # Apply filters
```

```
    if student_id:
        query = query.filter_by(user_id=student_id)
    if subject_id:
        query = query.filter_by(subject_id=subject_id)
    if date:
        query = query.filter_by(date=date)

    # Get paginated results
    attendances = query.order_by(
        Attendance.date.desc(),
        Attendance.time.desc()
    ).paginate(page=page, per_page=per_page)

    # Get all students and subjects for filter dropdowns
    students = User.query.filter_by(is_admin=False).all()
    subjects = Subject.query.all()

    return render_template('admin/attendance_history.html',
                           attendances=attendances,
                           students=students,
                           subjects=subjects,
                           selected_student=student_id,
                           selected_subject=subject_id,
                           selected_date=date)
@app.route('/student/attendance-history')
def student_attendance_history():
```

```
    page = request.args.get('page', 1, type=int)
    per_page = 10

    # Get all attendance records for the current user, newest first
    attendances = Attendance.query.filter_by(user_id=current_user.id)\
        .order_by(Attendance.date.desc(), Attendance.time.desc())\
        .paginate(page=page, per_page=per_page)

    return render_template('student/attendance_history.html',
                           attendances=attendances,
                           user=current_user)
@app.route('/admin/generate-report')
def generate_report():
    if 'user_id' not in session:
        return redirect('/login')

    user = User.query.get(session['user_id'])
    if not user or not user.is_admin:
        return redirect('/unauthorized')

    subjects = Subject.query.all()
    students = User.query.filter_by(is_admin=False).all()
    report_data = []

    for student in students:
        student_report = {'student': student.name, 'roll_no': student.registration_no, 'subjects': [],
'overall_percentage': 0}
```

```
        total_present = 0
        total_classes = 0

        for subject in subjects:
            attendances = Attendance.query.filter_by(user_id=student.id, subject_id=subject.id).all()
            total = len(attendances)
            present = len([a for a in attendances if a.status == 'Present'])

            percentage = round((present / total * 100), 2) if total > 0 else 0
            student_report['subjects'].append({
                'subject': subject.name,
                'present': present,
                'total': total,
                'percentage': percentage
            })

            total_present += present
            total_classes += total

        student_report['overall_percentage'] = round((total_present / total_classes * 100), 2) if
total_classes > 0 else 0
        report_data.append(student_report)

    return render_template('admin/report.html', report_data=report_data)

@app.route('/static/<path:filename>')
def static_files(filename):
```

```
    return send_from_directory(app.static_folder, filename)
@app.errorhandler(404)
def page_not_found(e):
    return render_template('errors/404.html'), 404


@app.errorhandler(500)
def internal_server_error(e):
    return render_template('errors/500.html'), 500
if __name__ == '__main__':
    app.run(debug=True)
```

EXPERIMENT REQUIREMENTS

HARDWARE REQUIREMENTS

RAM: 8 GB

HARD DISK: 512 GB

Processor: core i5 or i7

Architecture: 32-bit or 64-bit

SOFTWARE REQUIREMENTS

i) Technologies used: Visual studio, python with OpenCV (Open-Source Computer Vision), Tkinter GUI for frontend.

ii) Operating System: Windows 7 and above or Linux based OS or MAC OS.

Python:

Python is an interpreted, high-level, general purpose programming language created by Guido Van Rossum and first released in 1991; Python's design philosophy emphasizes code Readability with its notable use of significant Whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. Python is dynamically typed and garbage collected. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python provides libraries like OpenCV, NumPy for image and video processing. These libraries use pre-trained models to detect the presence of faces in images or video frames. Once a face is detected, the system can crop the face and use it for recognition. It provides a simple way to store data in databases. The attendance system can store the attendance data in a database or export it to a file for further analysis. It provides libraries like PyQt and Tkinter for developing Graphical User Interfaces (GUIs) for the attendance system. These allow developers to create intuitive interfaces for users to interact with the system.

NumPy:

NumPy is an open-source library available in Python, which helps in mathematical, scientific, engineering, and data science programming. It is a very useful library to perform mathematical and statistical operations in Python. It works perfectly for multi-dimensional arrays and matrix multiplication. It is easy to integrate with C/C++ and Fortran. For any scientific project, NumPy is the tool to know. It has been built to work with the N-dimensional array, linear algebra, random number, Fourier transform, etc.

OpenCV:

OpenCV (Open-source computer vision) is a library of programming functions mainly aimed at real- time computer vision. Originally developed by Intel, it was later supported by willow garage then Itseez (which was later acquired by Intel). The library is cross platform and free for use under the open-source BSD license. OpenCV supports some models from deep learning frameworks like TensorFlow, Torch, PyTorch (after converting to an ONNX model) and Caffe according to a defined list of supported layers. It promotes Open Vision Capsules. which is a portable format, compatible with all other formats.

OpenCV's application areas include:

- 2D and 3D feature toolkits
- Ego motion estimation
- Facial recognition system
- Gesture recognition
- Human–computer interaction (HCI)
- Mobile robotics
- Motion understanding
- Object identification
- Segmentation and recognition
- Stereopsis stereo vision: depth perception from 2 cameras

- Structure from motion (SFM)
- Motion tracking
- Augmented reality

To support some of the above areas, OpenCV includes a statistical machine learning library that contains:

- Boosting
- Decision tree learning
- Gradient boosting trees
- Expectation-maximization algorithm
- k-nearest neighbour algorithm
- Naive Bayes classifier
- Artificial neural networks
- Random forest
- SVM

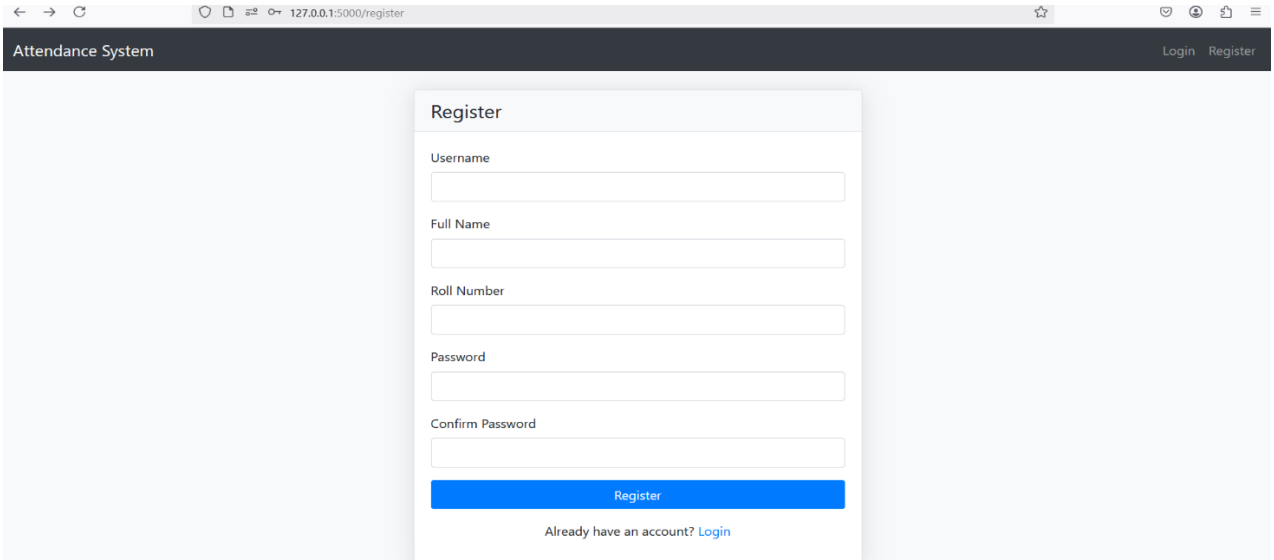
Tkinter:

Tkinter is the standard GUI library for Python. Python when combined with Tkinter provides a fast and easy way to create GUI applications. Tkinter provides a powerful object-oriented interface to the Tkinter GUI toolkit. Creating a GUI application using Tkinter is an easy task. All you need to do is perform the following steps –

1. Import the Tkinter module.
2. Create the GUI application main window.
3. Add one or more of the above-mentioned widgets to the GUI application.
4. Enter the main event loop to take action against each event triggered by the user.

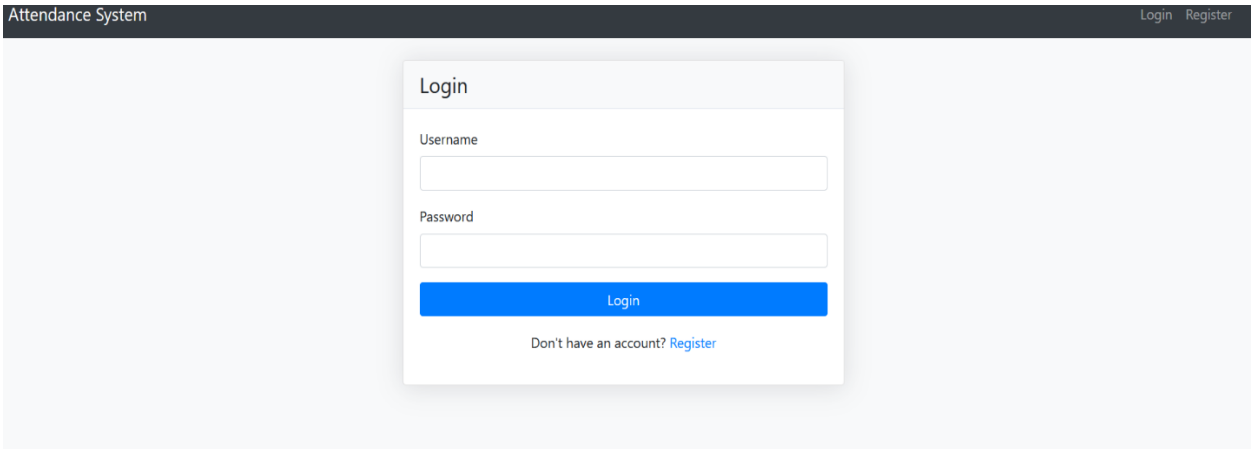
RESULT AND ANALYSIS

Student's Register Page

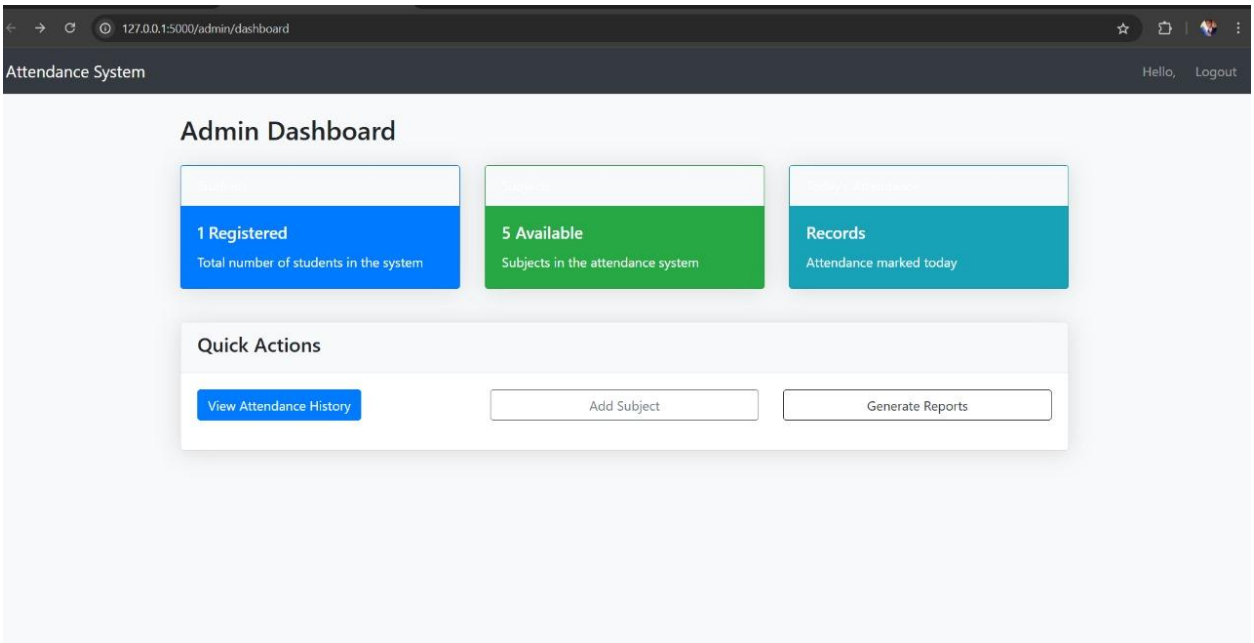


(Fig-8) System GUI

Login Page



(Fig-9) Login Page



(Fig-10) Admin Dashboard

ADVANTAGES

The Face Recognition Attendance System offers numerous advantages over traditional attendance methods. Firstly, it provides a **contactless and hygienic solution**, which is especially important in the post-COVID era where minimizing physical touch is crucial. Unlike fingerprint or manual sign-in systems, users only need to face the camera for automatic recognition and attendance logging. Secondly, it significantly **reduces time and administrative effort**, as attendance is marked within seconds without manual intervention. This helps in improving productivity in educational institutions and workplaces. Another major advantage is its **accuracy and reliability**—the system eliminates chances of proxy attendance or buddy punching, ensuring that only authorized individuals are marked present. It also supports **real-time monitoring and reporting**, making it easier for administrators to generate attendance records and analyze data. Additionally, the system is **scalable** and can be implemented across large organizations without requiring complex infrastructure. It uses open-source tools like Python and OpenCV, making it a **cost-effective solution** with minimal hardware requirements. Furthermore, with proper data protection practices, it ensures **secure storage of biometric information**, maintaining user privacy. Overall, the system enhances efficiency, accuracy, and security in attendance management.

LIMITATION

While the Face Recognition Attendance System offers many benefits, it also comes with certain limitations that must be considered. One significant limitation is **accuracy under varying environmental conditions**. The performance of facial recognition can be affected by poor lighting, extreme brightness, shadows, and background interference. For instance, in low-light environments, the system may fail to detect or correctly recognize a face. Similarly, **occlusions**, such as hats, masks, sunglasses, or long hair covering the face, can hinder the accuracy of recognition. Another limitation is the **variation in facial expressions and aging**. Changes in facial features due to emotions, makeup, or aging over time may reduce the accuracy of the system if the facial data is not updated periodically.

Moreover, **hardware dependency** is another concern. Although the system can work with standard webcams, low-resolution or low-quality cameras may compromise detection speed and recognition precision. Additionally, the system requires **sufficient computational resources** for real-time processing, especially when dealing with large datasets or high-resolution images, which may not be feasible on low-end machines.

Privacy and ethical concerns are also noteworthy. Storing and processing biometric data involves risks related to **data breaches and misuse** if proper security measures are not in place. Compliance with **data protection regulations**, such as GDPR, is essential but can be complex to implement. Lastly, the system may **struggle with high levels of user diversity**, such as recognizing people from different ethnic backgrounds or with similar facial features, which could lead to bias or misidentification.

FUTURE PROSPECTS

The future prospects of the Face Recognition Attendance System are highly promising, driven by ongoing advancements in artificial intelligence, computer vision, and biometric technologies. As facial recognition algorithms continue to improve in accuracy and efficiency, the system can be enhanced to function reliably even under challenging conditions such as poor lighting, partial occlusion, or varying facial expressions. In the near future, integration with **cloud computing and IoT (Internet of Things)** will enable real-time attendance monitoring across multiple locations, making it ideal for large organizations and educational institutions with distributed campuses.

Moreover, the system can be integrated with **mobile applications and cloud databases**, allowing administrators and users to access attendance records remotely. Future versions may also incorporate **multi-factor authentication**, combining facial recognition with RFID cards or mobile OTPs to further strengthen security. Additionally, the application of deep learning models like **convolutional neural networks (CNNs)** can drastically enhance recognition accuracy and speed. There is also potential for incorporating **emotion detection, thermal screening**, and **automated alerts**, transforming the system into a comprehensive smart surveillance and management tool. With ongoing improvements and wider adoption, face recognition attendance systems are set to become a standard solution for secure, efficient, and automated attendance tracking.

CONCLUSION

In conclusion, the Face Recognition Attendance System presents an innovative and efficient solution for automating attendance in educational and organizational settings. By leveraging facial biometrics, it offers a contactless, accurate, and time-saving alternative to traditional methods. Despite a few limitations related to environmental factors and data privacy, the system's advantages far outweigh its challenges. With ongoing advancements in artificial intelligence and computer vision, the system holds great potential for future enhancements in security, scalability, and performance. Ultimately, it contributes to smarter, more reliable, and technology-driven attendance management.

REFERENCE

1. Verma, S., et al. (2023).

AI-Based Face Recognition Attendance System.

International Journal of Innovative Research in Management, Engineering and Technology (IJIRMPS), 2(2024), 230598.

https://www.ijirmps.org/papers/2024/2/230598.pdf

2. Ashwin Rao. (2022).

AttenFace: A Real-Time Attendance System Using Face Recognition.

arXiv preprint arXiv:2211.07582.

https://arxiv.org/abs/2211.07582

3. Bao-Thien Nguyen-Tat, Minh-Quoc Bui, & Vuong M. Ngo. (2024).

Automating Attendance Management in Human Resources: A Design Science Approach Using Computer Vision and Facial Recognition.

arXiv preprint arXiv:2405.12633.

https://arxiv.org/abs/2405.12633

4. Virtue Market Research. (2023).

Face Recognition Attendance System Market Size (2023 – 2030).

https://virtuemarketresearch.com/report/face-recognition-attendance-system-market

5. **Ray, D. (2025).**

A Face Recognition Based Attendance System with Geolocation and Real-Time Action Logging.

https://www.researchgate.net/publication/388572615_A_Face_Recognition_Based_Attendance_System_with_Geolocation_and_Real-Time_Action_Logging

6. **Jagadale, S. (2023).**

AI-Based Attendance System using Face Recognition.

https://abhivruddhi.mituniversity.ac.in/wp-content/uploads/2023/08/Research-Paper_Sachin-Jagadale.pdf

7. **Authors Unknown. (2023).**
Face Recognition Attendance System.

https://www.ssgmce.ac.in/uploads/UG_Projects/cse/202324/Project%20Report%20Gr.%20No.%2010_2023-24.pdf

8. **Authors Unknown. (2025).**
Automatic Student Attendance System Based on Face Recognition.

https://pubs.aip.org/aip/acp/article/3157/1/070005/3344648/Automatic-student-attendance-system-based-on-face

9. **Rao, A. (2022).**
AttenFace: A Real-Time Attendance System using Face Recognition.

https://arxiv.org/abs/2211.07582

10. **Journal of Informatics and Visualization (JOIV).**
An Automated Face Detection and Recognition for Class Attendance.

https://joiv.org/index.php/joiv/article/view/2967